2. Bölüm

Temel Yazılım Kavramları

2.1 Düşük Düzey Programlama

1642 Yılında *Blaise Pascal* tarafından icat edilen Pascalline Hesap Makinesi, eldeli toplama, ödünç almalı çıkarma yapıyordu.

1671 Yılında *Gottfried Wilhelm Leibniz* tarafından icat edilen Leibniz Çarkı, toplama, ödünç almalı çıkarmanın yanında bölme, çarpma ve karekök işlemlerini yapıyordu.

1801 Yılında *Joseph Maria Jacquard* dokuma tezgahlarında desenleri oluşturmak için **delikli kartlar** (punch card) kullanmayı icat etmiştir. Sonradan bilgisayara veri girdisi sağlamak ve sonuçların çıktısını göstermek için kullanılacaktır.

1830 Yılında *Charles Babbage* fark makinesi olan analitik makineyi icat etmiştir. Buhar gücü kullanan bu makinenin mantıksal işlem birimi, veri depolama birimi, giriş çıkış üniteleri bulunuyordu. *Augusta Ada Byron*, Babbage ile beraber çalışmıştır. Byron, analitik makinenin bir dizi matematiksel işlemi gerçekleştirebildiğini fark etti ve bu makineyi Bernoulli sayılarını üretmek için kullandı. Bu sayıların hesaplanması için yazdığı algoritma tarihteki ilk bilgisayar programı olarak kabul edilir.

Augusta Ada Byron, **ilk programcı** olarak kabul edilir.

1854 Yılında *George Boole* tarafından ikili sayı sistemini geliştirmiş bugün bilgisayarlarımızın kullandığı **bool cebiri** üzerine çalışmalar yapmıştır.

1890 Yılında *Herman Hollerith* tarafından delikli kartlarla bilgilerin yüklenebildiği ve bu bilgiler üzerinde toplama işlemlerinin yapılabildiği bir elektro mekanik araç geliştirdi. ABD'nin 1890 nüfus sayımında başarılı biçimde kullanıldı. Hollerith başarılı olunca bir şirket kurdu ve daha sonra üç firma birleşerek 1924 yılında adını **IBM** olarak değiştirdi.

1941 Yılında *Konrad Zuze* Z3 isimli elektrik motorları ile çalıştırılan mekanik bir bilgisayar yaptı. Bu (Z1, Z2, Z3 ve Z4 serisi) program kontrollü ilk bilgisayardır.

1944 Yılında *Howard Aitken*, IBM ile iş birliği yaparak **MARK-I**'i yaptı. Bilgiler, delikli kartlarla veriliyor ve sonuçlar yine delikli kartlarla alınıyordu. Saniyede 5 işlem yapabiliyordu. Boyutları,18 m uzunluğunda ve 2,5 m yüksekliğinde idi. İnsan müdahalesi olmadan sürekli olarak, hazırlanan programı yürüten ilk bilgisayar idi ve tam olarak elektronik bir bilgisayar değildi.

1943-1945 Yılları arasında *Konrad Zuze*, ilk yüksek düzey programlama dili olan Plankalkül'ü, Z1 bilgisayarı için geliştirmiştir.

Emir kodlarını (instruction code) ve sembolik isimleri (mnemonic) içermeyen programlama dilleri, **yüksek düzey programlama dili** (high level programming language) olarak adlandırılır. Yüksek düzey programlama dilleri çoğunlukla İngilizce sözcükleri kullanır.

1946 Yılında *John Von Neumann* önderliğinde Pensilvanya Üniversitesinde **ENIAC** (Elektronik Sayısal Hesaplayıcı ve Doğrulayıcı) isimli sayısal elektronik bilgisayar tamamlandı. Askeri amaçla üretildi ve top mermilerinin menzillerini hesaplamak için kullanıldı. Neumann tarafından geliştirilmiş bu mimari (Stored-program computer and Universal Turing machine § Stored-program computer), günümüzdeki bilgisayarlarda hala kullanılmaktadır. 18,000 adet elektronik tüp kullanılan ENIAC; 150 kW gücünde idi ve 50 ton ağırlığıyla 167 m^2 yer kaplıyordu. Saniyede 5.000 toplama işlemi yapabiliyordu. MARK-I bilgisayarından bin kat daha hızlıydı.

Yukarıda anlatılan bilgisayarlar, bir önceki bölümde anlatılan emir kodu (instruction code) veya sembolik isimlerle (mnemonic) yazılan programlar ile programlanmaktadır. Emir kodu ve sembolik isimlerle yapılan ve bilgisayar mimarisini bilmeyi gerektiren programlamaya **düşük düzey programlama** (low level programming) olarak adlandırılır. Daha çok elektronik ve bilgisayar mühendisleri ya da sistemi tasarlayan matematikçiler bu programlama türünü kullanır.

Düşük düzey programlamanın yapılabilmesi için hem işlemcinin hem de hafıza ve işlemciye bağlanan diğer bileşenlerin nasıl çalıştığı ve tasarlandığını çok iyi bilinmesi gerekir.

2.2 Genel Amaçlı Yüksek Düzey Diller

Sembolik isimler veya makine kodu yerine bildiğimiz konuşma diline yakın talimatlarla metin olarak yazılan programlama dilleri, **yüksek düzey programlama dilleri** (high level programming language)olarak adlandırılır. Yani emir kodu (instruction code) ve sembolik isim (mnemonic) kullanmayan programlama dilleri yüksek düzey programlama dilleridir.

Yüksek düzey dillerde **talimatlar** (statement) olarak yazılan programlar yine kendine has derleyiciler tarafından makine diline çevrilir. Her talimat, genellikle birden fazla emirden (instruction) oluşur.

Yüksek seviyeli bir dilde talimatlar, yapılması gerekeni kısa ve net olarak belirten **mantıksal satırlardır** (logical sequence).

1954 Yılında **FORTRAN** (FORmula TRANslation), *John Backus* liderliğinde IBM 704 için geliştirilirdi. FORTRAN, `READ`, `WRITE`, `IF` gibi 32 farklı talimata (statement) sahiptir. İlk genel amaçlı, yüksek düzey bir programlama dilidir. Aşağıda üç kenarı teyp ünitesinden alınan bir üçgenin alanını hesaplayan FORTRAN-II programı verilmiştir [1]. Görüldüğü üzere talimatlar kısa öz ve anlaşılırdır.

```
C AREA OF A TRIANGLE - HERON'S FORMULA
C INPUT - CARD READER UNIT 5, INTEGER INPUT
C OUTPUT -
C INTEGER VARIABLES START WITH I,J,K,L,M OR N
READ(5,501) IA,IB,IC
501 FORMAT(3I5)
IF (IA) 701, 777, 701
701 IF (IB) 702, 777, 702
702 IF (IC) 703, 777, 703
777 STOP 1
703 S = (IA + IB + IC) / 2.0
AREA = SQRT( S * (S - IA) * (S - IB) * (S - IC) )
WRITE(6,801) IA,IB,IC,AREA
801 FORMAT(4H A= ,I5,5H  B= ,I5,5H  C= ,I5,8H  AREA= ,F10.2,
$13H SQUARE UNITS)
STOP
END
```

1956 Yılında ticari amaçlarla kullanılabilen ve seri halde üretimi yapılan ilk bilgisayar **UNIVAC-I** oldu. UNIVAC, giriş-çıkış birimleri manyetik bant idi ve bir yazıcıya sahipti. UNIVAC 1951-1959 arasında üretilen bilgisayarlarda vakum tüpleri kullanıldı. Bu tüpler bir ampul büyüklüğünde, çok fazla enerji harcamakta ve çok fazla ısı yaymakta idiler. Veri ve programlar manyetik teyp ve tambur gibi bilgi saklama araçlarıyla saklandı. Veriler ve programlar bilgisayara delgi kartları ile yükleniyordu.

1959 yılı sonrasında üretilen bilgisayarlarda transistörler (yaklaşık 10 bin adet) kullanıldı. Bu yıl ve sonrasında transistör içeren **ikinci kuşak bilgisayarlar** hayatımıza girmiştir.

2.3 Arapsaçı Kod

1955-1959 Yılları arasında **FLOW-MATIC B0** (Business Language Version 0) Dili, *Grace Hopper* tarafından UNIVAC-I için geliştirildi. İngilizceye benzeyen ilk yüksek düzey programlama dilidir. Aşağıda B0 programlama dilinde örnek bir program verilmiştir [2].

```
(0)  INPUT INVENTORY FILE-A PRICE FILE-B ; OUTPUT PRICED-INV FILE-C UNPRICED-INV
FILE-D ; HSP D .
(1)  COMPARE PRODUCT-NO (A) WITH PRODUCT-NO (B) ; IF GREATER GO TO OPERATION 10 ;
IF EQUAL GO TO OPERATION 5 ; OTHERWISE GO TO OPERATION 2 .
(2)  TRANSFER A TO D .
(3)  WRITE-ITEM D .
(4)  JUMP TO OPERATION 8 .
```

[1] https://github.com/scivision/fortran-II-examples/blob/main/herons_formula.f
[2] https://gist.github.com/marcgg/f9f2da31a973ba319809

```
(5)  TRANSFER A TO C .
(6)  MOVE UNIT-PRICE (B) TO UNIT-PRICE (C) .
(7)  WRITE-ITEM C .
(8)  READ-ITEM A ; IF END OF DATA GO TO OPERATION 14 .
(9)  JUMP TO OPERATION 1 .
(10)  READ-ITEM B ; IF END OF DATA GO TO OPERATION 12 .
(11)  JUMP TO OPERATION 1 .
(12)  SET OPERATION 9 TO GO TO OPERATION 2 .
(13)  JUMP TO OPERATION 2 .
(14)  TEST PRODUCT-NO (B) AGAINST ZZZZZZZZZZZZ ; IF EQUAL GO TO OPERATION 16 ;
OTHERWISE GO TO OPERATION 15 .
(15)  REWIND B .
(16)  CLOSE-OUT FILES C ; D .
(17)  STOP . (END)
```

Görüldüğü üzere bu tarihlere kadar yazılan programlarda bir sürü `GO TO` ya da `JUMP TO` talimatı (statement) bulunmaktadır. Bu durumda programın ne yaptığı konusunda fikir yürütmeye çalıştığımızda **okunaklılık oldukça azdır**. İşte okunaklılığın az olduğu bu program kodlarına **arapsaçı kod** (spaghetti code) adı verilir. Günümüzde ne yaptığı kolay anlaşılamayan kodlar için bu kavram çokça kullanılmaktadır.

İşin doğası gereği **makine diline çevrilecek tüm kaynak kodlar gözle okunmalıdır!**.

1958 Yılında **LISP** programlama dili, *John McCarthy* önderliğinde *Steve Russell* tarafından geliştirilmiş ikinci yüksek düzey programlama dilidir;

```
(format t "Adinizi Giriniz: ")
(setq name (read))
(princ "Merhaba ")
(write name)
```

1959 Yılında **COBOL** (Common Business-Oriented Language) programlama dili, CODASYL komitesi tarafından İngilizceye benzer ikinci yüksek düzey bir dil olarak geliştirilmiştir;

```
IDENTIFICATION DIVISION.
PROGRAM-ID. GREET.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 USER-NAME PIC A(30).

PROCEDURE DIVISION.
DISPLAY "Enter your name: ".
ACCEPT USER-NAME.
DISPLAY "Hello, " USER-NAME "!".
STOP RUN.
```

1958 Yılında daha sonra **ALGOL** (ALGOritmic Language) adını alan **IAL** (International Algebraic Language) programlama dili geliştirilmiştir. Bu dil daha sonra ortaya çıkan birçok dile önderlik etmiştir. Bu dil ile birlikte kod blokları (code block), iç içe fonksiyon (nested function), değişken kapsamı (scope) ve dizi (array) kavramları programcının hayatına girmiştir. ALGOL dili, barındırdığı özellikler sebebiyle, kendisinden sonra ortaya çıkan bütün programlama dilleri için bir esin kaynağı olmuştur;

```
BEGIN
FILE F (KIND=REMOTE);
EBCDIC ARRAY E [0:11];
REPLACE E BY "HELLO WORLD!";
WRITE (F, *, E);
END.
```

1958 Yılındaki FORTRAN diline diline çıktı üretmeyen fonksiyonlar için `SUBROUTINE`, değer üreten fonksiyonlar için `FUNCTION`, `CALL` ve `RETURN` özellikleri eklenmiş ve **FORTRAN-II** adını almıştır.

Benzer şekilde 1958 ile 1969 yılları arasında ALGOL diline çıktı üretmeyen fonksiyonlar için `PROCEDURE` ve `BEGIN-END` arasında kod bloğu özellikleri eklenmiştir.

1966 Yılındaki FORTRAN-IV diline `INTEGER`, `REAL`, `DOUBLE PRECISION`, `COMPLEX` ve `LOGICAL` **veri tipleri** (data type) ile Mantıksal `IF`, aritmetik (üç yollu) `IF` ve `DO` döngüsü talimatları (statement) eklenmiştir.

1964 yılında Dartmouth College'da John G. Kemeny ve Thomas E. Kurtz tarafından geliştirilen **BASIC** (Beginner's All-purpose Symbolic Instruction Code) öğrenilmesi ve kullanılması kolay olacak şekilde tasarlanmış bir programlama dilidir. Bilimsel olmayan alanlardaki öğrencilerin ve yeni başlayanların bilgisayar kullanabilmesini sağlamak amacıyla İngilizce benzeri komutlarla tasarlanmıştır. Karmaşık donanım detaylarını gizleyerek programcının mantığa odaklanmasına izin verir. Kodlar genellikle satır satır gerçek zamanlı olarak çalıştırılır, bu da hataların anında bulunmasını kolaylaştırır. 1980'li yılların başında kadar ev bilgisayarlarının (Apple II, Commodore 64, IBM PC gibi) varsayılan dili haline gelmiştir;

```
10 REM A simple program to greet the user
20 CLS ' Clears the screen (command might vary by implementation)
30 PRINT "What is your name? ";
40 INPUT NAME$ ' The '$' indicates a string variable
50 PRINT "Hello, "; NAME$; "!"
60 GOTO 30 ' Jumps back to line 30 to run the program again
70 END ' Terminates the program
```

2.4 Değişken

Değişkenler (variable) aslında cebirde kullandığımız değişkenlerdir. Cebir (algebra) işlemlerinde doğrudan problemde verilen sayıları değil, onları temsil eden değişkenleri kullanırız.

$$3x^2 + 2x + 10$$
$$c^2 = a^2 + b^2$$
$$c = 2\pi r$$

Yukarıdaki cebirsel ifadelerin ilkinde bir polinom, ikincisinde ise bir dik üçgenin kenar uzunlukları arasındaki ilişki verilmiştir. Bu ifadelerde x, a, b ve r **bağımsız değişken**, c ise **bağımlı değişkendir**. Örneklerdeki 3,2,10,2 ve π ise **sabit** (constant) sayılardır.

Değişkenler, bilgisayarlarda bir miktar bellek bölgesini işgal eder ve bu bellek bölgesinde çeşitli biçimlerde tutulurlar.

Tam sayılar, **ikili** (binary) sayı olarak bellekte tutulur. Tam sayıyı ifade eden ikili sayının en anlamlı biti, işaret (sign) biti olarak adlandırılır ve bu bitin değeri `1` ise sayı negatif, `0` ise sayı pozitiftir. Tablo 2.1'de en çok kullanılan tam sayılara ilişkin bellek miktarı ve sayı sınırları verilmiştir.

Bit Sayısı	Bellek Miktarı	Sayı Sınırları
8 Bit	1 bayt	-127 ile +128 arasında veya 0 ile 255 arasında
16 bit	2 bayt	-32.767 ile +32.768 arasında veya 0 ile 65.535 arasında
32 bit	4 bayt	-2.147.483.648 (-2^{31}) ile +2.147.483.647 ($2^{31}-1$) arasında veya 0 ile 4.294.967.295 ($2^{32}-1$) arasında
64 bit	8 bayt	-9.223.372.036.854.775.808 (-2^{63}) ile 9.223.372.036.854.775.807 ($2^{63}-1$) arasında veya 0 ile 18.446.744.073.709.551.615 ($2^{64}-1$) arasında

Tablo 2.1: Tam Sayılar ve Sayı Sınırları

Gerçek sayılar, kayan noktalı sayı (floating point number) olarak biçimlendirilir. Kayan noktalı sayı kavramı ilk olarak 1914 yılında Leonardo *Torres Quevedo* tarafından Charles Babage'ın analitik makinesine kullanılmak üzere yayınlanmıştır.

$$12345 = 1234,5 \times 10^1 = 123,45 \times 10^2 = 12,345 \times 10^3 = 1,2345 \times 10^4 = 0,12345 \times 10^5$$

Yukarıda ondalık tabanda verilen gerçek bir sayıya ilişkin kesir noktasının kaydırılması ile aynı sayı ifade edildiği bir örnek verilmiştir. Kesir noktası sola kaydırıldığında sayı, 10 ile çarpılmış, sağa kaydırıldığında ise sayı 10'a bölünmüş olur. Kayan noktalı sayının üs kısmı dışında kalan kısmı taban veya **kesir** (fraction) olarak adlandırılır. Tabanın kesir noktası sağında kalan hanelerine ise **mantis** (mantissa) adı verilir. Tablo 2.2'de en çok kullanılan kayan noktalı sayılara ilişkin bellek miktarı ve sayı sınırları verilmiştir[3].

1938 yılında *Konrad Zuse*, ilk ikili (binary) programlanabilir mekanik bilgisayar olan Z1'i yapmıştı. Bu bilgisayar, 7 bitlik işaretli üs, 17 bitlik anlamlı değer ve bir işaret biti içeren 24 bitlik ikili tabanda kayan noktalı sayı kullanmıştır.

[3] IEEE Computer Society (2019-07-22). IEEE Standard for Floating-Point Arithmetic. IEEE STD 754-2019. IEEE. pp. 1–84.

Bit Sayısı	Bellek Miktarı	Sayı Sınırları
32 bit	4 bayt	$1,2 \times 10^{-38}$ ile $3,4 \times 10^{+38}$ arasında **tek hassasiyetli** gerçek sayılar; en soldaki 1 bit işaret biti, sonraki 8 bit üs ve geri kalan 23 bit ise kesir olarak kullanılan ikilik tabanda sayılardır.
64 bit	8 bayt	$2,3 \times 10^{-308}$ ila $1,7 \times 10^{+308}$ arasında **çift hassasiyetli** gerçek sayılar; en soldaki 1 bit işaret biti, sonraki 11 bit üs ve geri kalan 52 bit ise kesir olarak kullanılan ikilik tabanda sayılardır.

Tablo 2.2: Kayan Noktalı Sayılar ve Sınırları

Tam sayılarda olduğu gibi **kayan noktalı sayılar da bilgisayarlarda ikili (binary) sayı tabanında tutulur**. Örneğin ondalık tabanda 50,25 sayısı ikili tabanda $1.1001001x2^5$ olarak ifade edilir.

Program yazılırken tanımlanacak değişkenlere ilişkin bellek ihtiyacı programcı tarafından belirlenir. Programcı değişkene tahsis edilecek bellek miktarına ve tutacağı içeriğe göre **veri tipi** (data type) belirler. Değişkenlerin veri tipine kısaca **değişken tipi** (variable type) adı da verilir. Veri tiplerine verilen adlar dilden dile değişir; Örneğin PASCAL dilinde 8 bitlik tam sayıya `bayt` denilirken, C dilinde `char` adı verilir. Programcı tarafından kullanılacağı amaca göre belirleyeceği veri tipinde istediği kadar değişken tanımlayabilir.

2.5 Fonksiyon

Matematikte **fonksiyon** (function), bir kümenin bir veya daha fazla kümenin elemanını tek bir kümenin elemanına eşleyen **ilişkidir** (relationship).

Girdi	İlişki	Çıktı
1	x2	2
2	x2	4
3	x2	6
4	x2	8
...		

Tablo 2.3: İki İle Çarpma Fonksiyonu

Girdi olan bağımsız değişkenler ile çıktı olan bağımlı değişken arasındaki bu ilişki, matematikte aşağıdaki şekilde **ifade** (expression) edilir.

$$f(x) = 2x$$

Burada f, fonksiyon anlamında x ise girdi anlamında olup **parametre** olarak adlandırılır. Böylece tabloda verilen fonksiyon tablosuna her girdiyi yazmak zorunda kalmayız. Bazen fonksiyona aşağıdaki örneği verilen şekilde bir ad verilmez;

$$y = 2x$$

Bazı durumlarda ise girdi daha detaylı tanımlanır;

$$x \in R, f(x) = 2x + 33$$

Bu fonksiyon, reel sayı kümesindeki her x elemanını, hesaplanan f(x) değerine eşleyen bir ifadedir. Yani x=1.0 argümanını f(1.0)=5.0 reel sayısına eşler. Kısaca fonksiyon 5.0 reel sayısını hesaplayıp **döndürür** (return).

$$g(a, b) = 2a + b$$

Bu fonksiyon ise reel sayı kümesindeki her a elemanı ile b elemanını, hesaplanan g(a,b) değerine eşleyen bir ifadedir. Yani a=1.0 ve b=1.0 argümanlarını g(1.0,1.0) fonksiyonu yine reel sayı kümesinden 3.0 reel sayısına eşler. Kısaca bu fonksiyon, 3.0 reel sayısını hesaplayıp döndürür (return).

Örneklerdeki x, a ve b parametre olarak adlandırılır. f(1.0) ifadesindeki 1.0 ise parametrenin o an andığı değeri belirtir ve **gerçek parametre** (actual parameter) ya da **argüman** (argument) olarak adlandırılır.

$$f(3.0) + g(3.0, 2.0) + f(2.0) + g(1.0, 3.0)$$

Fonksiyonlar ifade olarak tanımlandıktan sonra yukarıdaki gibi tekrar tekrar **çağrılabilirler** (call).

2.6 Yapısal Programlama

1970'li yıllara kadar her ne kadar çıktı üretmeyen fonksiyonlar için `SUBROUTINE`, birden fazla çağrı noktası olabilen `COROUTINE` ve `FUNCTION` kullanılıyor olsa da hala `GO TO` ve `JUMP TO` ifadeleri kullanılmakta ve kafa

karışıklığına devam etmekteydi.

> 1968 Yılında *Edsger W. Dijkstra* önderliğinde **`GO TO` ve `JUMP TO` ifadeleri zararlı olarak ilan edilmiştir.**

1970 Yılında *Niklaus Wirth* tarafından **Pascal** dili geliştirildi. Bu dil ilk geliştirilen yapısal programlama dilidir. **Yapısal programlamada** (structural programming) verileri taşıyan veri yapıları (değişkenler ve diziler) ile bunu işleyen kontrol yapıları (`if`, `for`, `do`, `repeat`) birbirinden ayrılmıştır. Pascal, kendi kendini çağıran **özyinelemeli** (recursive) fonksiyonlar ile değişkenlerin adreslerini tutan **göstericileri** (pointer) desteklemektedir[4]. Aşağıda iki sayıyı kullanıcıdan alan ve konsola toplamı yazdıran bir program örneği verilmiştir;

```
program ToplamaIslemi;
var
sayi1, sayi2, toplam: integer; { Değişken tanımlama bölümü }
begin { Ana fonksiyon başlangıcı }
    { Kullanıcıdan veri alma }
    writeln('Birinci sayiyi giriniz:');
    readln(sayi1);
    writeln('Ikinci sayiyi giriniz:');
    readln(sayi2);
    { Hesaplama }
    toplam := sayi1 + sayi2;
    { Sonucu ekrana yazdırma }
    writeln('Girdiginiz sayilarin toplami: ', toplam);
end. { Ana fonksiyon bitişi }
```

> Yapısal programlamada programlama, fonksiyonların birbirini çağırmasıyla yapılır. **Yapısal programlamada veri ile veriyi işleyen yapılar birbirinden ayrıldığından arapsaçı programlamanın aksine okunaklılık çak yüksektir.**

2.7 Yapısal Programlama Genel Çerçevesi

1. Programın başladığı yeri belirten bir **ana fonksiyon** (main function) tanımlanır. Kodlaması yapılacak işi birbirini çağıran fonksiyonlar olarak bu ana fonksiyonda yazarız.
2. Her fonksiyonda ilk önce **veri yapıları** (data structure) tanımlanır. Veri yapısı;
   (a) En basit veri yapısı `char`, `integer`, `float` ve `double` gibi **ilkel veri tiplerinden** (primitive data type) olabilen **değişken** (variable),
   (b) İlkel veri tiplerinden meydana gelen **dizi** (array),
   (c) İlkel veri tiplerinin birkaçından veya dizilerinden bir araya gelmesiyle oluşan **yapılar** (structure),
   (d) Ya da dinamik olarak yani çalışma anında (run-time) birbirine bağlanmış verilerden oluşan **bağlı liste** (linked list) olabilir.
3. Her fonksiyonda tanımlanan veri yapılarının ardından bu yapıları işleyecek **kontrol yapıları** (control structure) tanımlanır. Kontrol yapıları bir veya daha fazla veya iç içe `if`, `if-else`, `switch`, `while`, `do-while`, `for`, `continue`, `break`, `goto` ve `return` **talimatlarından** (statement) oluşur.

Yapısal programlamanın en çok uygulandığı Pascal dilinde ana fonksiyon nokta ile biten `BEGIN-END.` bloğudur. C dilinde ana fonksiyon, `main()` fonksiyonudur.

C Programlama Dili, *Dennis M. Ritchie* tarafından Bell Laboratuvarlarında UNIX işletim sistemini geliştirmek için geliştirilen genel amaçlı, üst düzey bir dildir. C, ilk olarak 1972'de DEC PDP-11 bilgisayarında çalıştırılmıştır. *Ken Thompson* tarafından geliştirilen B adlı daha eski bir dilden gelişmiştir. 1978 yılında *Brian Kernighan ve Dennis Ritchie*, günümüzde K&R standardı olarak bilinen C'nin ilk kamuya açık kılavuzunu hazırlamışlardır.

C programlama dili; Öğrenmesi kolaydır, **yapısal programlama dilidir**, hızlı ve verimli programlar üretir, assembly dili desteği ile **düşük düzey programlamayı** (low level programming) da destekler ve standart başlık dosyaları sayesinde çeşitli bilgisayar platformlarında derlenebilir. Bu özellikleri nedeniyle neredeyse tüm işletim sistemlerinin çekirdeği C dilinde yazılmıştır ve sistem dili olarak da adlandırılmaktadır. Yukarıda verilen Pascal yapısal programının C dilinde kodlanmış hali aşağıda verilmiştir;

```
#include <stdio.h>
int main(){
```

[4] Wirth, Niklaus (1976). Algorithms + Data Structures = Programs. Prentice-Hall. p. 126

```
    // Değişken Tanımlama
    int sayi1, sayi2, toplam;
    // Kullanıcılardan veri alma
    printf("Birinci Sayıyı Giriniz:");
    scanf("%d",&sayi1);
    printf("İkinci Sayıyı Giriniz:");
    scanf("%d",&sayi2);
    // Hesaplama
    toplam=sayi1+sayi2;
    //Sonucu ekrana yazdırma
    printf("Girdiğiniz Sayıların Toplamı:%d",toplam);
    return 0;
}
```

2.8 Nesne Yönelimli Programlama İhtiyacı

C++ programlama dili, C diline yapılan nesne yönelimli programlama OOP (object oriented programming) eklentileri yapılarak geliştirilmiş bir programlama dilidir. 1979 senesinde bir Danimarkalı bilgisayar bilim adamı olan *Bjarne Stroustrup*, sonradan C++ olarak bilinecek olan **C with Classes** üzerinde çalışmaya başladı ve 1985 yılında ilk sürümünü yayınladı.

Yapısal programlamada talimatlar (statements) art arda koda yazılarak programlama yapılır ve programların neler yaptığı bu talimatlar izlenerek anlaşılabilir. Talimatların zincirin halkaları gibi birbirinin peşi sıra yazılarak yapılan programlamaya ise emreden programlama (imperative programming) paradigması adı verilir. Bu paradigmaya sahip diller, yazılımı yapılacak sürece ilişkin nelerin yapılacağını değil, işin nasıl yapılacağını belirtirler. Emreden paradigmanın bir örneği olan yapısal programlamada zamanla aşağıdaki problemler ortaya çıkmıştır;

- Kod ne kadar büyürse, modüllere ayırma imkânı olmasına rağmen, fonksiyonlar arasındaki bağımlılık da o kadar artar. Bir fonksiyonun parametrelerindeki değişiklik, onun kullanıldığı tüm yerlerde değişiklik gerektirir. Değişim yönetimi (change management) çok zordur ve uzun zaman alır.
- Hata durumunda yapılacak işlemlere ilişkin kod ile iş sürecini gerçekleştiren kod iç içedir. Aynı fonksiyonun bazı durumlarda hata, bazı durumlarda ise değer döndürmesi mümkündür. Çoğu durumda hataların izini sürmek (error handling) işi zorlaşır.
- Yapısal programlamada, gösterici (pointer) kullanımında erişilmesi istenmeyen bellek bölgelerine erişilmesi halinde istenmeyen program davranışları ortaya çıkar. Kodun güvenli (safe code) olarak çalıştığının incelenmesi çok zordur.
- Sürekli olarak benzer projelerde aynı kodları tekrar yazmak (duplicate code) gereklidir.
- Emreden paradigmanın burada belirtilen sebepler başta olmak üzere çeşitli problemleri bulunmaktadır. Bu nedenle 1980'li yıllarda birçok yazılım projesi başarısız (fail) olmuştur.

Bir yazılım; Kullanıcı ihtiyaçlarının karşılanamaması, Öngörülen bütçenin aşılması ve Zamanında teslim edilememesi durumlarında başarısız olur. Yapısal programlamadaki bu sıkıntıları gören yazılımcıların imdadına Simula ve Smalltalk programlama dillindeki yaklaşım yetişmiştir. Bu diller oldukça yavaş olmasına rağmen getirdiği çözümler oldukça yenilikçiydi.

Smalltalk, 1972 yılında Xerox Park şirketinde *Alan Kay* önderliğinde *Dan Ingalls*, *Adele Goldberg*, *Ted Kaehler*, *Diana Merry*, *Scott Wallace* ve *Peter Deutsch* tarafından geliştirilmiş bir dildir. Smalltalk dilinde nesneler (object) temel yapıtaşlarıdır. Nesneler;

- Durum (state) ve davranışlara (behavior) sahiptir.
- Programlama, nesnelerin birbirlerine ileti göndermesiyle (message-passing) yapılır.
- Nesnelerin kendi ya da bir başka nesnenin davranış ve durumlarını öğrenebilme yeteneği yani yansıma (reflection) özelliği vardır.

Yapısal programlamanın içinde olduğu emreden paradigmanın (imprative programming) aksine, **nesne yönelimli programlama** OOP (object oriented programming paradigmasında progamlama, nesnelerin birbirlerine ileti göndermesi bakış açısıyla yapılır.)

2.9 Düşün ve Çöz

- **Düşün**: Arapsaçı kod (spaghetti code) kavramının yapısal programlama ile nasıl çözüldüğünü ve fonksiyonların bu süreçteki rolünü tartışınız.
- **Düşün**: Bir fonksiyon, 'ne yapılacağını' mı yoksa 'nasıl yapılacağını' mı tarif eder? Bu ayrımı gerçek bir

örnek üzerinden (örneğin bir sıralama algoritması) nesne yönelimli ve yapısal programlama perspektifinden karşılaştırın.

- ◇ **Düşün**: Assembly dilinde yazılan bir program, yüksek seviyeli dilde yazılana göre neden daha hızlı çalışabilir? Ancak neden artık sistem programlaması dışında tercih edilmemektedir?
- ◇ **Düşün**: Değişken (variable) kavramı ile değişmez (literal) arasındaki temel fark nedir? Bir programda değişken yerine yanlışlıkla değişmez kullanılırsa hangi tür hatalar ortaya çıkabilir?
- ◇ **Düşün**: Yapısal programlamada alt programlar (procedure) ile nesne yönelimli programlamadaki yöntemler (method) arasındaki kavramsal farkı açıklayın. İkisi aynı şey midir?
- ◇ **Düşün**: Simula ve Smalltalk dillerinin nesne yönelimli programlamaya katkısını araştırın. Bu dillerin C++'a etkisi neden 'devrimci' olarak nitelendirilir?
- ◇ **Düşün**: Algoritma tasarımında yalancı kod (pseudocode) kullanmanın, doğrudan kod yazmaya başlamaya göre hata payını nasıl azalttığını açıklayınız.
- ◇ **Düşün**: Modüler programlama yaklaşımı, büyük bir yazılım projesinde farklı mühendislerin aynı anda çalışmasını nasıl kolaylaştırır?
- ◇ **Çöz**: Bir öğrencinin vize ve final notlarını temsil eden iki değişken tanımlayınız. Bu verilerin işlenerek ortalama bilgisine dönüşme sürecini basit bir algoritma ile gösteriniz.
- ◇ **Çöz**: Bir sayının asal olup olmadığını kontrol eden bir algoritmanın **akış diyagramını** (flowchart) çiziniz.
- ◇ **Çöz**: Kullanıcının girdiği üç sayıdan en büyüğünü bulan algoritmanın adımlarını mantıksal bir sıra ile listeleyiniz.